

Matthew Brown: "Real-Time Active Inference for Adaptive Robotic Control"

58:19

hello welcome back we're here with
Matthew Brown discussing realtime active
inference for adaptive robotic control
so put away the memories that you
haven't had yet and let's talk about
robotics thank you Matt yeah thank you
Daniel yeah this is going to be a bit of
a change in Direction although that was
that was a really great talk I'm looking
forward to reviewing a lot of the talks
from the last um last day um but that
was pretty pretty amazing from China um
so yeah I'm Matt uh great great to be
here um super excited to share updates
about what we're doing um just to give a
little bit of background on myself uh I
I've been working in uh artificial
intelligence for a little over 20 years
um including time in the defense
industry writing AI for video games
things like Halo Bioshock Splinter Cell
those franchises and over the last 10
years been working in more serious AI uh
particularly uh in deep reinforcement
learning and now in active inference uh
notably uh built the first commercially
available deep reinforcement learning
platform which was bought by Microsoft
in 2018 is now the autonomous systems
platform there um currently I'm at a
startup at TH for I began about five
years ago uh thought forge's technology
started as a personal research project
back in 2007 that never really ended
exploring um biologically plausible

alternate approaches to machine learning
uh and in the over the last couple years
have been applying it to Adaptive
real-time control for robotics so that's
what I'm going to be going through today
um just a Qui quick uh brief agenda on
on what what I'll be talking about and
go through a quick uh well not so quick
I'll go through an introduction a lot
about the highle concepts uh then dig
into uh briefly the history of the
project uh and the implementation uh and
then go into some of the more
interesting topics in depth here um
Network self models embodiment
synchronization a lot of the tools that
we're using um to get these um robots to
do interesting things and then um spent
some time talking about uh what we've
been doing with robotics so show some
videos uh perhaps um and just uh explain
a little bit about the direction that
we're going over the next uh couple
years so I'm not going to spend very
long on this slide as is probably
redundant uh you know as the topic tour
I will just briefly emphasize the last
point on the slide here um which is
about um how the free energy principle
provides a unifying framework that
explains how adaptive Behavior Uh
perception and action can emerge from
the drive to maintain a stable internal
state in a constantly changing
environment this is going to be a a
theme throughout the talk so I figured
point that out uh and this one as well I
probably not going to go into too much

but it's worth um noting the last two points here uh about organisms don't passively respond to sensory input but also actively shape their environment through actions that fulfill their predictions about their sensory experience and then also about how active inference explains how uh organisms achieve complex self-directed Behavior by combining sensory processing processing with active exploration um this is also uh going to be a common theme throughout the uh the talk um now we're going to go a little bit deeper uh into what we're specifically using for our robotic uh models uh and the roots in cybernetics uh um in particular the relationship between the free energy principle active inference and cybernetics which might be obvious to many people um uh at the Symposium um cybernetics here being pioneered in the 1940s um also describes self-regulating systems that adapt to their environments uh and also kind of uh views agents as directed systems seeking to maintain equilibrium through feedback loops uh the goal here being to minimize free energy or surprisal in uh the FP and active inference uh and you know active inference kind of operationalizes this through continuous adjustment of beliefs and actions um and these cybernetic ideas are foundational I believe to how the free energy principle and active view organisms as systems that maintain their states within a predictable range uh and also resists

entropy and disorder by adjusting
internal and external variables um over
time now digging One Step deeper into
the approaches that we're taking here um
I'm going to talk a little bit about um
the cybernetics of Ross Ashby in
particular um uh cybernet Ross aspy he
formalized many key ideas um homeostasis
in particular as the process by which an
organism regulates internal conditions
uh within a narrow optimal range uh in
the face of environmental changes and
disturbance
um he's known for um these are probably
the two biggest things he's known for
the good regulator theorem and the law
of requisite variety uh the good
regulator theorem commonly is quoted is
every good regulator of a system must be
a model of that system um or more
accurately every good regulator must
contain a model of that system um and
with the law of requisite variety the
the common quote is only variety can
destroy variety and you know another way
of thinking about this is a system must
be as complex or more complex than its
envir in order to manage it effectively
or if if one system wants to effectively
uh manage or control another system it
must be at least as expressive and at
least as complex as the system is
controlling um in the active inference
framework an agent maintains homeostasis
by minimizing prediction error
effectively predicting and compensating
for uh environmental fluctuations uh so
this is how Ash's ideas uh really kind

of come into play here as's emphasis on adaptive regulation through internal models aligns very closely with active inferences process of continuously updating beliefs and taking actions to minimize prediction error and maintain uh desired States um Ash's view of homeostasis as a self-regulating feedback system is effectively a form of active inference both involve an agent using internal models to anticipate in counteract environmental changes uh preserving stability and adaptability through continuous self-correction uh and Ashby's law of requisite variety here highlights the you know a lot of the successes of active uh when active is successful uh and of maintaining ostatic here um relying on the system's capacity to generate a diverse and precise range of predictions and actions uh without sufficient variety of responses the system would fail to adapt leading to a breakdown in homeostasis and an increase of prediction error predictive error um uh to exemplify these ideas uh Ross as Ross asby constructed a device known as the homeostat this was uh back in the 1940s um this was a demonstration of learning as adaptation as as stability uh and he believed that behavior is adaptive it contributes to the maintenance of these essential variables uh and there's some various ways of looking at what he was doing here um some called it an isomorphism generating machine uh using feedback to generate distributed homeostasis via Ultra

stability and and I I'll get more into
that in a little bit um but that's kind
of the approach that we've taken here at
thorge uh and then there's kind of a
long description here from one of his
books about um what the homeostat was
and how to construct it um and all talk
a bit more about how do you construct a
homeostat in a in in a moment here um
here's kind of a quick highlevel
abstract view of home uh sorry Ashby's
original homeostat uh and this is
abstracted away in in a way that uh
reflects kind of modern views of um
mathematical networks here where you you
can define a network as a set of nodes
with connections and weights um and then
he he defined uh Ultra stability as kind
of the the the mechanism that powers the
race of State space B exploration uh
which is used as part of this law of
requisite variety uh and I'll talk uh in
the next slide a bit about that um the
network as a whole creates a dynamic
attractor Set uh as a kind of
self-consistent self-reinforcing uh
model uh and the network can be seen as
kind of a network of reciprocal
constraints um the original the original
homeostat only had four nodes here um
constructed from recycled World War II
bomb parts so this is pre- digital
computer era uh and this uh there a
diagram here kind of conceptually
explaining you know how how it worked um
the original H homeostat itself I think
is I think at the University right
now but the the the um description of of

how it actually works is uh somewhat abstract uh I we used uh at ffor we used a lot of his um technical journals which were released by his family I Believe In online in um in the last 15 years or so um to kind of get more detailed um information about the properties and how they're constructed to get a more accurate uh reconstruction of the homeostatic um briefly I'm just going to talk a little bit about Ultra stability

Ultra stability is the property of the individual node in this network that enables um the group property of homeostasis um the ultra stability here uh is talked about in in Ashby's books and papers um so I I I don't think it's necessary to go through a deep deep analysis of it um but it the ultra stability mechanism is kind of the the local the local mechanism within a node that drives the active process of self-organization and criticality across the network uh and it's it's core to the function of how the homeostat works uh and it is uh constructed by a double feedback loop with a low-level feedback loop that performs like a linear transformation then a highle feedback loop um that updates or changes that linear transformation uh over time um this is just uh I love I I like concluding this this this letter uh this was a letter from um Alan Turing to Ross Ashby uh way back in when they were kind of debating on whether or not to go with analog Computing or or digital Computing here being uh Ashby being supportive of

the analog Computing Paradigm uh and uh
this letter from tiing is basically
saying that his uh his computer the AC
was more General and more Universal than
Ash's computer because uh you could
Implement Ashby's computer on turing's
machine um and it's just kind of a fun
little piece of History here I I like
including it um just to provide some
context about the time when Ashby was
kind of considering these ideas uh and
why they were kind of lost to time in a
lot of Senses at least within the you
know Computer Science World um and I
also like to bring this up because uh I
think it's the first at at least in my
mind it's the first time um that we
really started conflating uh computation
with cognition here uh and where where
tiing had a universal comp computation
machine uh while Ashby was kind of
envisioning like a a universal uh
adaptation machine you know something
closer to cognition uh and it feels very
similar to a lot of the debate we're
seeing today uh with whether or not deep
learning machines are intelligent or you
know if they can get to AGI uh I think
that a lot of the conflation of
computation and cognition goes all the
way back to the origins of the analog
versus digital debate uh within computer
science um so the next question is how
do you actually build a homeostat uh and
I took turing's advice uh and simulated
we built a simulation of the homeostat
and the reason why simulation is useful
as opposed to like an analytical

solution is because of the chaotic nature of U of how these systems reach equilibrium um uh and I used to have a quote here from ILO P but um essentially you can't account for a lot of the chaotic interactions in the systems uh with loops algebraically uh the circularities and the the the the cha the chaos that's necessary for for these to stabilized require that you do this iter and that means um building a a a a simulation that you can kind of step forward uh and on the Fly generate the the the uh chaotic um interactions essentially uh as opposed to analytically um there's there's lots of references here with um you know the three body problem um uh a mapping of a line onto itself this this kind of concept of uh uh deterministic systems when you once you have three determin deterministic systems interacting with each other uh you inevitably get chaotic behavior um that is kind of leveraged as the creative uh creative energy that drives the the um the learning process in this kind of a system um the techniques that we use to actually simulate this uh and this is oops getting ahead of myself here um it come from a field known as software radio uh we started out by uh going through Ashby's technal journals uh and his writings about how the homeostat was constructed uh and we started out by building out uh accurate analog level digital simulations um and then from there we kind of abstracted

out all the mechanism specific Machinery
in the analog systems that are not
necessary for the actual function of
ultra stability and homeostasis so kind
of going from this kind of very very
precise analog system to uh kind of
abstracted implementation um of these
four nodes um once functional uh and
this was simple enough to kind of
uh further abstract into more kind of
modern-day Network Theory
implementations where we can kind of
build out very very large networks
similar to how it's done with deep
learning uh that can be connected into
arbitrary Network topologies as opposed
to the original for node Network here uh
and again you just to kind of reiterate
here the network as a whole can be seen
as a self model um each node in this
network is effectively trying to predict
its neighbors as it seeks equilibrium uh
and the the local Ultra stability
process Tunes anodes local connections
and the resulting positive and negative
feedback loops uh that uh that Define it
uh and these overlapping Loops become
self-reinforcing or self-repairing over
time once once you reached kind of um
convergence with these feedback loops uh
the final self-producing set of Loops is
a kind of self- model where each node is
trying to predict all the others and so
once all the nodes kind of reach a
global consensus uh this model this can
be Ed selfconsistent and uh reaches
homeostatic equilibrium um this gener
this can also be seen as kind of like a

generative model uh composed of emergent self-stabilizing homeostatic feedback loops uh and these Loops uh are maintaining this homeostatic equilibrium in a critical State and this is kind of an important note for why it is incredibly sample efficient which is going to be very important for robotic applications here um the idea here is that maintaining this homeostatic equilibrium at a critical State uh makes it very sensitive to prediction error you know the smallest predictive error uh can trigger a a global structural change you know um uh so you don't need to back propagate with a large data set in order to change the the the model uh the behavior of the model um a single misprediction can effectively trigger a cascading uh set of of redefinitions where the whole you know the whole network destabilizes and then Recon converges on a new attracting set Um this can also be seen as a form of synchronization and we'll get into that uh a little bit later um and the prediction error reaching zero kind of marks the establishment of an of a new adaptive synchronization manifold uh and the synchronization first starts with immediate neighbors uh at each node and then spreads through the entire network uh as a set of like compound hierarchies of feedback loops um looking at the time here

I sometimes I can I could show actually uh I could switch screens and show demonstrations of this but maybe I'll

I'll share some of I I'll wait a bit later to do uh demonstrations of the original homeostat or kind of larger homeostat um just so that we're not switching screens too much um in case that's useful um all right now we're getting into some of the more interesting stuff Le uh less talking about history uh of cybernetics um so the the homeostatic Network itself seems like a lot of work uh in terms of you know um there's there's a lot going on there and it's not connected to an environment it's just learning itself in coming reaching to convergence without kind of any um sensory or motor actions so um we have this model that can adapt to its internal structure to reach homeostatic equilibrium through a kind of self- synchronization process next step is how do we actually get this to drive uh Behavior Uh and this took you know a couple years to kind of work out the engineering uh conceptually it's pretty simple um but it will get very complicated very quickly um the short version is that the sensors and the motors uh um basically we Define sensors of motors uh and they Define how an agent is embedded into an environment uh and the network topology separately kind of defines the Adaptive capacity of the homeostatic agents model uh the sensor definitions themselves here um are are uh kind of defined externally from the model itself uh they provide a mechanism to influence the behavior of the system by providing priors in the form of

preferred sensory States or preferred environmental States uh and these act as the dynamic priors uh kind of externally find uh Dynamic set points for the homeostatic adaptation and these sensors can be seen as input attracting sets which can even be modified in real time or animated in real time to generate you know different kinds of behavior from the same model um at runtime without having to change the model structure or anything like that uh the motor definitions then provide you know kind of critical affordances for how the agent can modify the environment to help provide a basis for future sensory predictions um and the states of the sensors become the preferred outcomes of the motors through the this reaching equilibrium State and the model's Behavior ends up being biased as desired so this is how we get it to do useful behavior in a robot for example um from something that is seemingly just reaching equilibrium and I guess one important or Salient note here is that in a real world environment the you know there it's reaching pure homeostatic equilibrium is is actually quite difficult unless you're um unless you're in very special circumstances um so to expand a bit more on that um the network diagram here is kind of an overly simplified version of our homeostatic network but meant to kind of visually explain how to think about these uh in a particular here um

embodiment of of of these models in kind of an environment here um coupling it coupling a model like this to an environment through sensors and Motors is kind of a form of embod here meaning there's a kind of a specific time place that this model is operating uh and there is um the there is kind of a temporal component here where um the the uh the model's sequence of sensory Behavior actually is is relevant you the the um the the inputs to the system are not independent which is kind of different than traditional what you would imagine back propagation based perceptrons how they function uh and one thing that's interesting about this uh and and and it's worth pointing out here is that there's no forward or backward no feed forward or feed backward um uh directions in this kind of network there's no front or back or top to bottom um the only thing you can really kind of uh Direction you can kind of take from a model like this is distance from sensors or Motors or or how close to the environment a particular node is um otherwise relative location is pretty much everything you know where as as an individual node where am I relative to the rest of the network uh and it's it's common for people to think of of machine learning models these days especially if you're used to to think about deep learning or traditional perceptron based back propagation models uh to think of it as a sequential flow of processing like uh I take in an input I perceive or

I do planning on that input and then I generate an action um that's not how these Network networks uh function uh instead um everything kind of happens all the time and everything's overlapping uh what I mean by that is um this is not a sandwich model I guess is is the term time simply advances and information moves from one node to another um all in parallel you know so in in traditional deep learning it's kind of an atemporal process where information goes from the inputs to the outputs instantaneously uh and uh there's no internal time unless you're working with recurrent neural networks and there are ways of adding a temporal aspect of it but the temporal aspect is really a fundamental feature of of these types of notes um what that means is um the the network itself the the homeostatic feedback loops come to be dependent on expected sensory signals u meaning the the deeper parts of the network are generating predictive signals forward to the sensors uh and so um in order for this to function you the the deeper parts of the model have to be actually actively predicting the sensory inputs that that are coming in um this I believe also means that it ends up becoming grounded in the embodied experience itself meaning the the homeostatic self-maintenance of this network is composed of environmental signals from the Environ from from The Real World or whatever whatever embodiment you you you've put this

model in um and as part of its kind of predictions and expectations meaning the the actual sensory experience is what what is being predicted and so what the models actually generated is um generating is is derived originally from this uh essentially embodied experience um the expectations and predictions for the consequences of motor actions are kind of embedded in the behaviors uh that are being generated embedded in the signals sent this uh to the sensors from deeper in the network at the same time as these kind of predictive signals are being sent to the to the sensory regions actions are being sent to the motors um and these actions at any moment are not based on the latest input but based on predictions about what the model believes is going on in the world world and how best to to kind of satisfy the sensory inputs or get the the model into a preferred state or a lower energy state or or lower surprisal State um these kind of these motor actions are accompanied in parallel by sensory predictions that are all moving kind of to the to the to the this the uh the I don't want to say the front of the network but the the abstraction that defines the embodiment you know to the sensors and Motors um in kind of in parallel um this is this may be familiar to people who are who are um have read about perceptual control theory and predictive processing and very much uh along those same exact lines in terms of what's going on here in terms of uh

actively generating predictions to counteract sensory uh sensory input um the mo the the motors here end up being kind of um a critical part of making those sensory the future sensory experiences um easier to predict or um as part of this whole process another way to look at this this homeostatic process is through uh synchronization um the the homeostatic process can be seen as generating an Adaptive synchronization manifold between the agent and the environment um based around minimization of prediction error basically uh minimizing the difference between what I'm predicting the environment is doing and what the environment is actually doing this is kind of a synchronization as prediction uh process uh where when the two systems are synchronized they can be thought of as predicting each other you know um you know anticipating the other individual nodes are using uh Ultra stability uh predict their neighbors and then when at equilibrium all the nodes are synchronized through these feedback loops uh the synchronization between the nodes spreads to whole network synchronization at this point uh ultimately uh this synchronization spreads across this this sensory and motor abstraction uh between the environment and and and uh an agent so you kind of get a um agent environment synchronization process uh through this um looking at another way uh these overreacting feedback loops interact and

synchronize with each other as part of the homeostatic self-maintenance as these feedback loops extend from Motors through an environment back into the sensors the model gains the ability to anticipate the consequences of its own actions uh and the the the capacity to synchronize internal states with something in the external World um you know without direct communication is is kind of critical to this the synchronization means that the internal States become uh uh they they they come to resemble internal states of the other system um when and when at this kind of synchronized homeostatic State it's also in this kind of critical State sensitive to local prediction errors um and this like I mentioned before this is kind of critical because that allows internally the model to to completely restructure itself or redefine how it predicts Century experience um at the very slightest misprediction um and uh this is this is very very critical for sample efficiency um uh with to contrast this with you know deep learning models or deep reinforcement learning models you need um a a very large amount of data to kind of change um The Domain the or the way in which the the models are behaving in the world while in these kind of models it can kind of change at the uh with the slightest slightest uh one piece of data can kind of trigger Global uh Global restructuring um so one interesting aspect here is that in order to maintain

a synchronization manifold the network must be able to converge to equilibrium faster than the environment can destabilize it and so this kind of goes back to um as's theory of uh the good regulator theorem or the law of requisite variety where the model has to be ex more complex than the environment is trying to to control um and has to be at least as expressive as the as the as the uh the environment is in terms of coming up with Solutions um and you know uh yeah actually I was going to go a bit more into synchronization uh and how once once the network is synchronized with an environment um once two elements internal to those systems are synchronized they can kind of coordinate without direct communication and this is kind of important to the whole process as so that the sensor and motors abstra don't have to be the direct link between uh an aspect of the model and an aspect of the environment once synchronized those two pieces can coordinate in terms of their behaviors will be coordinated without direct communication between the two um so now on to some more fun stuff um the killer application of all this is robotics uh other early Avenues we looked into is uh for example applying this for anomaly detection um where you can of use the network as a a thermometer for surprisal um the other way you can kind of use this applied to like modern deep learning is for like a model dri drift detection where you can kind of uh use the model to um detect

when a deep Learning System is being used on a uh if the input stream to a deep Learning System is being is diverging from the model uh domain so like if the sorry the the trading domain so if the data set you used to train it on is starting to differ from how it's actually being used uh this could kind of uh be useful for that um but really I feel like robotics is really the killer application for this especially when you consider the embodied nature of it um you know this the embodiment is kind of a requirement for how this model Works in terms of having to be embedded in an environment with uh this could be a virtual environment as well but it has to have some sort of uh sensory motor coupling um to something external to itself um also the the ability to kind of understand context over time and understand that um even for a sensory input that is the same as some other uh uh a sensory input at another time the the the things that have happened previous to it are relevant you know um and so building up this context awareness uh through interaction with environment is kind of critical to how our models work um but also very important for for robotics uh and then real-time adaptation um of the model being able to update the model in real time as actions are being performed um is kind of a really useful technique in robotics in order to enable many uh dexterity uh and and adaptation in the moment um that is missing from a lot of

robotic applications here uh and specific to deep reinforcement learning and deep learning ways of doing robotic control there's this uh field known as Sim toore and so uh Sim toore problems come from uh deep reinforcement learning is is where they're commonly referred to but it's where the models are sensitive to simulation Fidelity uh the idea being you've trained a model in some in simulation um and how the model behaves is very sensitive to the specifics of the the physics in that simulation or you know the actual Fidelity of that s uh that simulation and so moving to the real world often shifts the input domain uh and deep reinforcement learning and deep learning models are not usually robust for these kinds of changes uh unless you you drastically increase the amount of data through domain randomization uh which causes other problems such as the curse of dimensionality there's all sorts of solutions to the Sy the real problem for us we sidestep all this by leveraging real-time learning and adaptation so um when the model is in presented with something new or unusual in the moment we we don't we don't separate the the training phase from the deploy you know the the inference phase it it as it's inferring it's doing the updating and the learning um and this this kind of solves a lot of the centore problems um as the model is able to adapt to whatever differences uh happen in the real world that were not represented in

simulation um so this is a lot of re uh
a longwinded explanation of why we went
after robotics for applying these these
kinds of models um to get into more kind
of robotic side of things in terms of
technical detail in terms of how we're
actually implementing this um for a lot
of our interoperability between a lot of
different robots um the model itself
doesn't really care what kind of robot
you apply this to uh so you get a lot of
Hardware agnosticism where you can take
a model built for one arm uh the robot
arm and apply it to another robot arm
for example uh and the model will adapt
to any kind of kinematic differences
between the two that kind of thing um
this is a diagram of how we fit into
Ross Ross is the Robot Operating System
and it's it's the it's the open- source
uh
academic uh software framework that is
used uh across that supports a lot of
robots it's not used in a lot of
industrial settings um but it's
incredibly flexible in its ability to
experiment and try new things and so
we've kind of used this as our kind of
beach head uh robotic stack
implementation um these uh homeostatic
models are particularly small uh so they
only operate like the uh typically about
30k of memory so uh they they can
operate on edge devices you know no TPU
needed uh no Cloud needed uh in fact you
you want to run it as close to the robot
as possible to minimize latency in the
control um phase uh and just runs in in

a single CPU um because of our our implementation of ultra stability and this homeostatic networks are quite efficient uh model update times our sub millisecond so this enables us to um update the model and update the the uh that synchronization manifold um as a task is being performed you know uh We've clocked the networks being able to run up to 80 khz um but for most robotic control algorithms 500 hzt or th000 or 1 khz is typically fantastic and most robots don't support anything above that um and there's you know there's AI also some other interesting aspects to the mo the the the technical side of things where the models are small enough we're talking 30k here where um analytical explainability is possible where we can take a look at a robotics action and actually analyze the homeostatic Network to figure out um which sensory inputs or which you know which motor out uh which sensory inputs led to a particular decision um and because the models are small enough we can kind of use different kinds of network analysis approaches uh to figure that out uh on the more technical detail side in case it's useful the our models are built in C++ and we have a python wrapper so that gives us a lot of integration to capabilities to lots of different robots um which is is is useful and fun um so just going to go through some of the applications that we've done in the past uh and I could I could bring up a demonstration and and do it live uh

this this was a uh a use case we did uh for a company for uh adaptive inspection so they had a use case where uh they have autonomous vehicles going through um going through a field and they needed a robotic arm to kind of uh compensate for um movement or unexpected you know orientations and and configurations of of the of the uh autonomous vehicle base um without knowing ahead of time what it was going to encounter uh and so yeah this was kind of a a fun interesting uh pretty simplistic model that we built for them uh and you can see kind of a fun video here of it in our lab kind of tracking this this QR code um and uh going one step further in terms of uh of difficulty this was another use case um from a different customer um they had a challenge of um they they were seeing an injury with um turning high pressure gas valves and uh they wanted to automate this with robots the problem was that they never have a good accounting for what kind of valve or what kind of uh you know thing that needs to be turned what it was going to look like what the shape of it was going to be if it was going to be rusty and difficult to turn or if it was going to be covered in oil and slippery or if the edges were going to be worn off um so they needed something that can adapt to this high level of variability um uh in real time um without access to the internet because this was on you know remote locations uh and so we built this model for them um and you this is just kind of

a fun little video of it um we've turned this into an interactive demo that we've brought to conferences which is um kind of fun here this is uh this is just recorded on a cell phone you can tell it was it's it's real based on the production qualities um but this was that that that valve turning model that we we created before um except we've just kind of adapted it here so that we can you know show off people torturing it um in real time live as it tries to kind of turn these bolts of various shapes uh and sizes um you know some interesting things to note here the original model that we built for this um only took about uh 15 seconds to train three examples essentially uh and it was only it was only trained on a single um bolt shape here uh the square in fact uh and then what we do is we kind of show how it can kind of generalize to all the these other different cases here in real time without having seen any of these examples before uh and at the beginning of the video there you can kind of see on the screen um uh our our internal it's our internal debugger but it's a fun visualization because you can see the the homeostatic Network um learning in real time as you kind of move this um this uh panel of bolts around um I'm just stalling a little bit there we go so you can see those little those little squiggly lines there on on the on the screen um that's the that's kind of a view into the 30 of the nodes of the network kind of um updating in real time

as we're as we're kind of having it turn
these knobs
here um getting into the less
interesting stuff but just to kind of
show uh where we are as as a technology
um this is a platform we're building uh
currently this is used internally um to
develop new models uh and we have um
some some interesting tools here that
allow you to um effectively search
ranges of topologies to to experiment
with different kinds of uh Network
topologies for particular task um and
then see what the performance looks like
in in simulation and kind of iterate on
this uh eventually you'll also be able
to uh connect your robot to the platform
and download the model directly um so
that you can go straight from um
designing uh a goal specification we
call a goal specification but really
what is what you're doing is you're
defining those T Dynamic priors in the
sensor uh sensor definitions as part of
that embodiment uh definition uh and
then you go and you evaluate what
topology is optimal for the you for that
for that definition and then you can
deploy this directly onto your robot uh
in the real world and try it out
um next steps or the direction that
we're heading um is is kind of
interesting um right now a lot of the
models that we're building are very kind
of small and uh task specific um but
we're building them in a way that is
recomposed of building one model that
does a task like turning the valves this

that that task is actually composed of three smaller models and the here is we're building up a library of recomposed submodels uh so that we can kind of recombine them in uh in in simple ways to go after new tasks uh so the idea here is we can take um right now we only have a handful of submodels but over the next year or two're building out this larger library of submodels that can be recomposed to go after a much wider variety of tasks uh and um so this this diagram here is is meant to show how we're kind of looking at um building up this framework of Rec composable models uh eventually that platform you saw this one here um eventually will'll probably open this up for the public uh to go and compose their own models from these recomposing submodels uh and um then we can even string together you know many models together and and um have a framework for deciding when to use which model um because they're only about 30k of memory uh you can really kind of load up uh a robot with you know let's say a thousand models for example uh and you can kind of get a lot of generality just from um a collection of sub problems being solved individually in in a robust manner um so I went through this talk pretty quickly I fig I'll just end with some current work that we're looking at uh right now we're working on a um this is just kind of an early look in simulation of of an insertion task um we're working on uh what's interesting

about this is that we can't use vision
for a lot of this because once the
insertion process happens um the the arm
kind of includes the view of it so we're
incorporating uh four sensors uh so that
we can kind of Wiggle you know tight
fitting pieces into a um a housing for
example uh and you really kind of need
to build in um uh this kind of nuanced
uh feeling of you know wiggling
something into place this comes very
naturally to humans you know um in terms
of like filling your way through a
problem uh and so we're we're kind of um
working on building out the submodels
needed here for high frequency Force
torque sensors for wiggling uh things
into place um to to kind of broaden the
amount kind applications that we can we
can start attacking with this approach
um so that's that's kind of uh some
current work that we're working on um I
kind of blazed through that really
quickly I could show some live
simulation demos but um maybe questions
maybe it makes sense to just move on to
questions thanks Matt that was awesome
I'll ask some of the questions in the
chat maybe we'll we'll still have time
to to see the demo but that was that was
really
exciting um okay I'll start with black
cat who wrote very cool how would you
say
interoperability challenges change from
software to
Hardware that that's inter a question
there's different ways to think about it

um moving to software things I think get a lot easier um as Hardware is always hard uh and always time consuming um for us we're using Ross to get a lot of interoperability between different kinds of robots but Ross is nonoptimal for for a lot of lot of situations and so there is going to be some sort of low-level drivers that we we'll have to write that are kind of going to be Hardware specific but for the most part most robot uh uh manufacturers or oems they have an interface that is that provides some sort of joint control so the models right now are uh generating effort-based controls uh that we then convert into you know positional based controls because that's the most common most robots do not support effort-based controls um but pretty much all robotic arms in particular have some sort of joint space control uh interface so we can basically all we need is kind of a driver to translate the output of the model into uh joint Space controls for that particular robot um the other thing that's kind of interesting and you don't get this with deep reinforcement learning is that um we don't actually specifically tell the model um what each motor output does we we we do have we do specify how many motor outputs there are from the model um uh although I guess we could overestimate we can give it more more actions larger action space than it actually has and will just kind of learn to ignore or it will just there will

become no correlations between the unused actions and the sensory space so it ends up being unnecessary um but because we don't actually specify what each motor action is doing um it learns that uh meaning like it it will in the first couple uh of milliseconds of interacting uh with the it considers the robot part of the environment let me put it that way so in the first couple milliseconds of of controlling this robot it will build up those correlations of oh if I apply a positive effort on this joint it moves me away from my target or if I move apply a negative it gets me closer to it kind of figures out those correlations and as you know as a process of it learning how to get to equilibrium if that makes sense um and as a result of that we get a lot of um Hardware agnosticism so with that that valve turning model for example it was originally built for a different robot it was originally built for a he heavy six degree of Freedom arm um the customer ended up deploying it on actually a different heavy robot with with an additional actuator um and the model can kind of you know we basically added an output to it um but the model can kind of adapt to that situation there um for those videos you just saw and and at that conference we then took that same model and deployed it on a different manufacture uh with a six degree Freedom arm uh and it kind of just figured out you know what it needs to do differently uh that robot is

really nice because it fits in a suitcase so it's really great to travel with so from from the uh moving to software and especially moving to this particular way of looking at the interface um it buys you a lot of interruptibility bonuses that you don't get with other approaches um but there is still you're still dependent on the hardware orem's driver level interfaces um but generally those are all the same uh with they the interfaces are all the same even if spinning up the robot might have different

mechanisms awesome Six Degrees of ACM I'm I'm going to Cluster two questions together so erun asked how much of this is or will be open source and Alexander Robbia asked wonderful talk Matt I'm a huge fan of the homeostat as you know have you given thought to perhaps translating the homeostat to its core set of equations dynamics and contributing them to open-Source projects like NGC learn so others can appreciate its

value oh great question um and great great I'm glad to hear Alex is on the call um these are both great questions

um I I'm always torn in terms of like what to open source and what not to open source um I think eventually we are going to open source a lot of this um right now the intellectual property of of this approach is one of the things that is most valuable about our current business so

from a business perspective in the short term we're probably not going to open source a lot of this we are expecting to open up our platform to to a wider audience um over over time

um I and actually I should follow up with Alex I we have actually kind of distilled down um our implementation of the homeostat at least you know there's a lot of inter interpretations I want to say there have been five or six implementations of the homeostat over the last 80 years or so uh and they're all different which is kind of interesting uh there's a lot of room for interpretation but at least our implementation of it we have kind of distilled down to a relatively simple set of equations and they they they are very very similar to kind of the the um equations using the perceptron with a couple of very subtle changes the the subtle changes here having drastic effects one one is that we've added state to the to the to the node um which really changes how how you think about these things as perceptrons are generally stateless and so the adding of the state there kind of adds that temporal notion so even though it's a subtle change it has drastic implications for how how the actual model Works um but uh and actually I might I was trying to see if I have it as an extra slide here in in the top um I might follow up with Alex and and share uh share those the equations I'm I'm less concerned about um sharing kind

of you know the approach and and the and
the uh the uh equations for for our
definition of the hom stat just because
these are very very difficult to
implement uh and I would kind of Welcome
more ideas and activity in the space and
I need to take a closer look at ngtc
learn uh I've I've been I keep seeing uh
uh things being added to NGC learn uh
and it's it's relatively new package but
there's a ton in there uh and I really
kind of need to take a look at what's in
there um and if we do open source it
does make sense to maybe integrate that
as potentially an option in NGC learn um
a lot of it depends on whether or not
the interface is compatible with what
we're doing and I I say that not because
as a as a limitation of NGC learn but
more of as a limitation of how people
think about machine learning models as
kind of um most most Frameworks that are
built around deep learning um their
interfaces kind of assume uh a temporal
you know uh Behavior where each input is
kind of independent uh each input output
relationship is independent of the
others um but I think it's just it's a
matter of me just taking a look at NGC
learn and how it how it's structured and
how to actually interface with it but I
think if we do eventually open source um
the code um that might be the the best
Avenue to do it just because that way it
would be alongside a lot of this other
great ideas for nonb back propagation
based machine learning thank you Matt
and and also really appreciate like

sharing that Insight it isn't a uh open
and shut and just because we can open
source something doesn't mean we could
or could in that way or should tomorrow
or whatever happens to be so so hearing
your your well-worn experience about all
of this is is really insightful um okay
another question from Alex he wrote how
do you go about designing the structure
of your homeostat net is it a random
graph that you instantiate or some other
graph type type does it depend on the
robotic control
problem that is a great question and
just right off the St off the start I
I'll mention that it's unsolved like
it's something that we're still working
on um in terms of how we do it now um so
longer term I would love to start with
like a random graph and then have some
sort of um metric for evaluating what
graph is better than another graph for
particular use case and and we're
probably pretty close to there and then
we could have some sort of mutation
function and that that way you could
kind of evolve what the right to is for
a particular graph I think that actually
has a lot of promise and I'd really love
to get into that kind of stuff right now
um what we're doing is a half step in
that direction um we have found that the
topology has a big impact on how good
the model is at performing particular
task meaning you can take a random model
and then try to have it turn vs for
example um but often it'll do a pretty
poor job or um it'll be slow or it'll be

a little wobbly um and and so really kind of fine-tuning what the topology is for a particular task is critical if you actually want a really effective robotic control model uh and so way that we've been solving this is um for for specifically robotic control we've come up with a way of parameterizing the topology um and this kind of constrains the search process for what is finding the right topology and then what we do is in that platform you saw in this platform the primary thing that this platform is doing is providing an interface to do that topology search so in the back end here uh when you actually are trying to find what is the optimal topology for a particular use case what's actually going on is that it's using that um that kind of parameterized search space and doing doing a big search uh and so we're actually leveraging some Cloud compute here for this not a ton uh it's not particularly expensive the training process only takes about 15 seconds for per model so um so what we're we're using any scale which is built on an open source technology called Ray uh which is uh really good at spinning up let's say of CPUs at once uh and so the idea here is we can go and spin up uh 20,000 different topologies uh and then evaluate of those 20,000 which one is optimal for a particular use case and this process takes about five minutes uh and so uh we're trying to get that as as short as possible so that this is kind

of a compilation time interaction as opposed to you know training the model for a day and then coming back and seeing how it performs you can go and like explore 20,000 different topologies and figure out which one is optimal for your particular use case so that's our current approach which is kind of like a brute Force approach um but longer term I would love to get to that evolutionary uh approach where we kind of start with a random model maybe we can estimate what a good starting size is or maybe we have some sort of um uh some you know uh heuristic for deciding whether or not to strength the model or or grow the model um but you know the to contrast it with deep learning um this is one of the downsides of the approach is that the topology itself is uh is very critical for good good performance as opposed with deep learning you can often just go with a larger model and just give it more data and you can kind of get the same performance as a smaller model um I think this makes sense uh from a biologic biological standpoint in the sense that our our brains are not just randomly connected they are evolved over time you know uh and you can imagine how some brains might perform better at certain tasks than other brains um and so I think this this topology dependent behavior is not necessarily um problematic or or wrong uh this just something we have to deal with awesome I'll read a comment from Scott David this is as most emissions from Scott's

language model it's not necessarily a question so if you have a response scope for it and then I do have a question that that also connects with Ian and little interception so Scott wrote it is interesting to consider the hardware software question of application of active inference in light of the historical counter of Claude Shannon with Von noyman when Shannon was encouraged to apply the second law of Thermodynamics to his quantitative theory of information thermodynamics at the edge of tangible and intangible information domains the ubiquity of the expression of the second law results in it being a foundational Vector for interoperability including across tangible hard and intangible soft domains there's a lot there I might I might need to take a look at that in text form and and come up with a coherent response um textually um but there's a lot there um there's there are a lot of information you know Shannon level information constraints in the you're right in at the hardware level especially when you're talking about um update rates you know for example um the model the model as a part of its embodiment has a temporal aspect to it too in terms of um the frequency of updates of the model has to be consistent like you can't update it you know 100 times per second you know for one second and then a million times per second for the next second so there is um there are kind of interesting

information Theory do you know problems here with you know Nyquist's theorem like uh as the sensory inputs are coming in um are are you getting sensory inputs at a high enough frequency to actually make interesting inferences about them or is on the robotic side there are absolutely uh control constraints you know if you're only able to take an action once per second there's you're very limited in terms of what you can do with a robot as opposed to if you can take an action every 500 every you know 2 milliseconds uh there's a lot more nuance uh in in the feedback process that of in the control so there's absolutely some information theoretic stuff um on on that interface in that embodiment you know definition um for the most part I haven't I I don't want to say we've done anything particularly innovative on that side it's just these are all things that you kind of have to understand as part of like the constraints of of the implementation um but uh definitely send me the text for that uh that question and I'll try to come up with a more coherent and thorough response um there was a lot in there cool and the historical part was was was epic um okay here's a question so I am not sure which parts you listen to from prior maybe Matt give a first pass but I was really interested when you mentioned the difference between the position sensors and the effort sensors so how does that play out in

robotics and then like Ian to the interception
an introspection
angle in what ways do does our postural adjustment relate to
positional inference and effort
inference that's a fun question um
unfortunately I haven't feeling my answer is going to be not part not too interesting um in terms of we're not doing anything particularly Innovative here either um the the outputs of the model are almost unitless uh in the sense that they'll they'll kind of adapt to their embodiment in terms of like so we've kind of been interpreting them as efforts or like you can think of this as like torque controls um the for robotics this is problematic though because first of all like I want to say 90% of robots or 90 not five% of robots don't support torque based controls that requires there there are just a few robots out there that require that and for the most part they're they're confined to research applications um the reason but you know the reason why they exist and the reason why there's a lot of interest in it especially with deep reinforcement learning is is there's if you can go to effort-based controls or Torque based control contrs directly to the robot there's a lot of nuanced Behavior you can get you can get really kind of uh compliant Behavior where the robot's applying just enough Force to perform a Tas that way if you bump into it it's it's not going to hurt anything and

it'll it'll be compliant and you can kind of get a lot of nuanced Behavior with torque base controls but the reality is that 95% of robots don't support that they just support either velocity controls or or you know positional controls of some kind uh and so to to solve that you know we're we basically convert the effort output of the model into a relative positional change and then we basically integrate that relative positional change over time to get an absolute positional control um so not particularly interesting on that side but uh it is kind of a technical note that's that's a bit challenging on the robotic side of things um it does I you know I hadn't really thought about it enough from an active inference perspective in terms of like what does that mean for um you know the the attracting set that you're you're bringing and a lot of it is because it's it's somewhat disconnected from the sensory inputs like um you know you know what I mean like the positional controls end up impacting the environment in some way and then you know connecting the dots kind of indirectly that affects your sensory inputs um but they're the it's not like a direct it it's not like the the positional versus effort outputs of the robot are directly changing how our sensory experience happens it's more that like it might change the the effectiveness of or the capabilities of your robot uh to it to to alter the

environment if that makes sense um but
maybe I need to think a bit more about
that because that is kind of an
interesting aspect that I I hadn't
really considered
enough how does that play out in the
body Ian oh yeah that's really
interesting I've joined this
conversation late but thanks for
including me Daniel um so yeah I was
kind of listening to that with interest
and thinking
of you know how we as a human something
simple like the the concept of being
hungry hungry and angry at the same time
and how that affects our behavior when
you've got complete competing kind of um
uh interests or physiological variables
that you're trying to deal with and then
thinking about okay a robot something
simple to start off with like maybe a um
one of these robotic lawnmowers that
needs to go back to its docking station
to charge up for
fuel um so it you know it needs to end
the task it's doing and move back to its
docking station and charge up but as
robots get more sophisticated and
they're trying to please humans more but
there will be some kind of you know I
don't know some limit to its um
capabilities its internal processing or
power that's and maybe at some point it
will say it's you know absolutely have
to weigh up all of these and come to
some computation about what it needs to
do next to satisfy its own and this
honestly I think ties into some of the

stuff that Alex has been talking to about with like moral competition as well um if I'm GNA be honest the robot control bottles were making are far less complex than an actual biological entity like a human um but you're absolutely right because you're going to have all sorts of here right now we're kind of having these kind of first layer sensory inputs kind of defining the tele damic PRI but you can imagine like a hierarchy where you have like other models that are feeding outputs that feed into your motor controls because these motor controls are are kind of the the end point for for modifying the environment but you can imagine why you would be reaching for a a cup of water is is driven by some other network that's saying oh you're thirsty um and uh and so we just you know I'm just not nearly there in of complexity yet but I think you're right eventually we will be especially if you want to have robots that are fully self self-sufficient fully autonomous know when they need to repair themselves and then going out and Performing those repairs uh whatever that might be maybe it's charging um I think you're absolutely right I think that's the future uh longer term I love it thank you Matt great times till next time yeah likewise

bye e